

Project #3, PhotoViewer (Due: 6/30 at 11:59pm)

Objectives

To practice event handling, JSON, and fetch.

Academic Integrity

Please make sure you read the academic integrity section of the syllabus so you understand what is permissible in our programming projects. Any case of academic dishonesty will be referred to the University's Office of Student Conduct. Please don't post your code online where others can see your code.

Overview

For this project you will implement a photo viewer that allows you to look at photos available in a folder or specified via a JSON file. A video illustrating some of the functionality of the application can be found [here](#).

Grading

- (16%) Form elements
 - (5%) Text fields
 - (10%) Buttons
 - (1%) Message of photo displayed in disabled text field
- (3%) Style
 - (1%) Shadow for buttons
 - (1%) Page brown border (groove style)
 - (1%) Photo black border (double style)
- (5%) Arrow (lambda) function requirement
- (2%) document.querySelector() requirement
- (2%) All JavaScript code in the code.js file
- (1%) "use strict"; directive is at the top of the code.js file
- (3%) All CSS code in the style.css file
- (68%) System's functionality

The project's code distribution is available in your upstream. The PhotoViewer folder includes:

- PhotoViewer.html - This is the main page of the application.
- code.js - This file includes all the JavaScript.
- style.css - This file includes all the css.
- umcp - Folder with images.

Specifications/Requirements

Loading Images from Local Folder

1. When the "Load Photos" button is selected, photos in the range defined by the "Start Photo Number" and "End Photo Number" can be displayed. Your code should read the folder name, common name, start photo number, and end photo number. Based on the previous information, your code can create an array with the names of the photos that belong to the specified range. Each photo's name consists of the folder name, followed by the common name, the photo's number, and the ".jpg" extension. You can assume only .jpg files will be provided. Notice you don't need to add "/" (it is assumed the folder name will provide it).
2. After loading data, the message "Photo Viewer System" should be displayed in a <div> that is in the "Status" fieldset area. This message should be displayed in red color.
3. The first photo in the range is displayed when the "First Photo" button is clicked on.
4. The last photo in the range is displayed when the "Last Photo" button is clicked on.
5. Clicking on the "Next Photo" button displays the next photo. If the last photo in the range is being displayed, selecting the "Next Photo" button will display the first photo.
6. Clicking on the "Previous Photo" button displays the previous photo. If the first photo in the range is being displayed, selecting the "Previous Photo" will display the last photo.
7. Selecting the "SlideShow" button creates an animation where photos in the specified range are displayed, continuously. The photos will be displayed in the same order defined by the range. For example, if the range is 4 and 6 (assuming an umcp/ folder and a common name of college) the images umcp/college4.jpg, umcp/college5.jpg, umcp/college6.jpg will be continuously displayed in that order.
8. Selecting the "RandomSlideShow" button creates an animation where photos in the specified range are displayed, randomly. Any random approach is fine and you don't have to worry if the same photo is displayed twice in a row. Ideally all photos in the range should eventually appear, but if not, that is fine. For example, selecting a random value in the range and displaying the photo associated with that value is fine.
9. Selecting the "StopSlideShow" button stops the current animation that is in progress.
10. Clicking on the displayed image is equivalent to clicking on the "Next Photo" button (the next image should be displayed when you click on the image currently displayed).
11. The name (folder, common name, number, .jpg) of the photo currently being displayed should be rendered in a disabled text field that is in the "Status" fieldset area. The text "Photo Being Displayed: " should precede the disabled text field. The name of the photo should be displayed for iterations (e.g., clicking on the "Next Photo" button) and for slide shows.
12. After reading the start and end photo numbers, your code must check that the end number is larger than or equal to the start number. If that is not the case, the message "Error: Invalid Range" should be displayed in a <div> that is located in the "Status" fieldset area. This message should be displayed in red color. You can assume we will always provide numbers in the "Start Photo Number" and the "End Photo Number" text fields.

13. Iterating (e.g., selecting "Next Photo") or starting a slide show without first loading data is considered an error and the message "Error: you must load data first" should be displayed in the <div> that is in the "Status" fieldset area. This message should be displayed in red color.
14. The image "InitialImage.jpg" will be displayed when the page is initially loaded. The name of this image should appear in the disabled text field.
15. The speed of the slide show animations is one second.

Loading Images Using a JSON File

1. The source of photos can be a JSON file that has the format the following example file illustrates <https://egonzal5.github.io/cmsc335/images/imagesSet1.json>
2. The JSON file has an array with images information. We will use the "imageUrl" property to initialize the array used by the photoviewer.
3. We will not use the "description" property.
4. You can assume the JSON file has valid data (i.e., images exist).
5. The "Load JSON" button will initialize the array of photos to view.
6. You must use the fetch API to download the JSON file with photos data.

Additional Information

1. The functionality of the viewer (e.g., animations, next photo, etc) is the same regardless whether we are loading data from a local folder or by using a JSON file.
2. Using global variables (variables that are not part of any function) is fine. Define as many as you want, but be reasonable about their use (i.e., if something can be solved without a global variable, then don't use it).
3. Try to avoid code duplication.
4. Functions requirement
 - a. You can have as many functions as you want.
 - b. Your code should rely on at least one arrow (lambda) function. So we can identify the function during grading, add the comment `/* LAMBDA */` near the function.
 - c. Your code should use the `document.querySelector()` method at least once.
5. The PhotoViewer.html file cannot have any JavaScript (only including code.js using the script tag).
6. All your JavaScript code must be in the code.js file.
7. In the code.js file, you can use `window.onload = main;` to call a `main()` function that sets the event listeners and performs any other initialization.
8. All your css code must be in the style.css file.
9. You can assume that if the start photo number is less than or equal to the end photo number, the numbers define a valid range, with valid photos.
10. You can assume that all photos in the provided folder have the same common name and have increasing numbers.
11. Your code should work for different folders (not just umcp/) and different range values.
12. You can assume that users will stop a slide show (using the "StopSlideShow" button) before starting another slide show (random or not).

13. You can assume that after a slide show has been stopped, if the user wants to iterate over the photos, they must select the "Load Photos" button.
14. You can assume that the photo's folder and the common name values will be valid (i.e., a folder with the specified name exists in the current folder and there are photos with the specified common name starting at one (e.g., college1.jpg)).
15. The page does not need to look exactly as show in the video. As long as it is close you are OK.
16. The "Reset Form Values" will reset the form values.
17. You may not use any authoring tool (e.g., Dreamweaver, etc.) that generates the HTML/CSS/JavaScript code for you.
18. You may not use Bootstrap, jQuery, React, or any other type of library/framework.
19. You must implement this project by yourself.
20. Make sure you view your code using Chrome (the browser we will use to grade your project). We will NOT use LiveServer to grade your project.
21. You can use any HTML/CSS code we have provided in lecture examples.
22. You should validate your HTML/CSS, but we will not validate it during grading.
23. This project does not require XAMPP/MAMP as it does not rely on PHP. You can use the VS Live Server extension to run your code.
24. You can assume that during grading we will first load data (either photos in range or fetch data) before starting a slide show, random slide show or iterating over the photos.
25. You must use "use strict"; in your JavaScript code.
26. You must use defer (in the <script> tag) when including JavaScript code.

Submission

Please submit your code to your gitlab repo. Please remember that we will grade your last submission!